

Erste Version der Einleitung: Trade-offs der Virtualisierung auf x86-Systemen: Eine quantitative Analyse von Performance-Overheads und Side-Channel-Risiken

Maximilian Wagner
Studiengang IT-Sicherheit
Fachbereich Informatik und Medien
Technische Hochschule Brandenburg
maximilian.wagner@th-brandenburg.de

Zusammenfassung—Dieses Dokument enthält die erste KI-assistiert erstellte Version der Einleitung der wissenschaftlichen Hausarbeit zum Thema mikroarchitektonischer Ressourcen-Interferenzen in virtualisierten Umgebungen. Im Anhang sind der verwendete System-Kontext sowie der eingesetzte Prompt zur Nachvollziehbarkeit des Erstellungsprozesses dokumentiert.

I. EINLEITUNG

a) *Thematische Einführung*: Virtualisierung bezeichnet die softwaregestützte Abstraktion physischer Rechenhardware in eine oder mehrere logisch isolierte Ausführungsumgebungen, sogenannte Gastsysteme. Der Hypervisor vermittelt dabei zwischen den Gastsystemen und der physischen Hardware und gewährleistet, dass Gastsysteme weder den Adressraum noch die Ausführungskontrolle benachbarter Instanzen direkt adressieren können. Diese logische Isolation ist das tragende Sicherheitsprinzip der Multi-Tenancy-Architektur. Auf mikroarchitektonischer Ebene ist diese Isolation jedoch unvollständig: Ressourcen wie der Last Level Cache (LLC), die Hauptspeicherbandbreite und I/O-Subsysteme werden zwischen allen auf denselben physischen Kernen eingepflanzten Gastsystemen physisch geteilt. Der Hypervisor abstrahiert diese Ressourcen nicht weg, sondern macht deren gemeinsame Nutzung für das Gast-Betriebssystem transparent — und entzieht ihr damit gleichzeitig jede Kontrollmöglichkeit [1].

b) *Relevanz und Gefährdungspotenzial*: Die unkontrollierte Teilung mikroarchitektonischer Ressourcen begründet eine systemische Schwachstellenklasse, die in modernen Cloud-Infrastrukturen mit wirtschaftlich signifikantem Schadenpotenzial verbunden ist. Koh et al. zeigen, dass Virtualisierungstechnologien zwar funktionale Fehler- und Sicherheitsisolation bereitstellen, eine strikte Performance-Isolation jedoch strukturell nicht leisten können [1]. Die daraus resultierende Ressourcen-Konkurrenz manifestiert sich in zwei Angriffsformen: Erstens ermöglicht die unkontrollierte Verdrängung von Cache-Lines durch einen aggressiven Workload — den sogenannten Noisy-Neighbor — eine ungezielte Degradation der

Latenz und des Durchsatzes koexistierender Gastsysteme und damit einen mikroarchitektonischen Denial-of-Service. Zweitens erlauben deterministische Zugriffsmuster auf den Shared LLC über Techniken wie PRIME+PROBE die Rekonstruktion von Laufzeitverhalten über logische Isolationsgrenzen hinweg, was als Seitenkanal-Angriff klassifiziert wird [2]. Das National Institute of Standards and Technology (NIST) klassifiziert die unkontrollierte Ressourcenerschöpfung auf Hypervisor-Ebene als kritisches Plattform-Risiko und fordert hardwaregestützte Isolationsmechanismen als Gegenmaßnahme [3].

c) *Zielsetzung und Problemstellung*: Die referenzierte Literatur belegt die prinzipielle Ausnutzbarkeit geteilter mikroarchitektonischer Ressourcen auf Basis älterer CPU-Generationen. Für aktuelle Hardware-Architekturen besteht eine empirische Forschungslücke: Weder die hybriden Kern-Topologien (P-Cores/E-Cores) noch die hardwaregestützten Isolationsmechanismen moderner Intel-Prozessoren — insbesondere die Intel Resource Director Technology (RDT) mit Cache Allocation Technology (CAT) — wurden in diesem Kontext systematisch untersucht [1], [2]. Diese Arbeit verfolgt das Ziel, folgende Hypothese empirisch zu falsifizieren: *Historische Interferenz-Vektoren, die auf der gemeinsamen Nutzung des Last Level Caches beruhen, führen auf einer Intel Raptor Lake Architektur unter produktiven Hypervisor-Bedingungen (Proxmox VE) nicht mehr zu statistisch quantifizierbaren Isolationsbrüchen*. Die Forschungsfrage lautet entsprechend: Sind LLC-basierte Noisy-Neighbor-Interferenzen auf aktueller Hardware mit modernen Mitigierungen empirisch nachweisbar?

d) *Lösungsansatz*: Die Forschungsfrage wird durch einen empirischen Messansatz unter deterministisch kontrollierten Bedingungen adressiert. Anstelle einer rein theoretischen Analyse werden Interferenzeffekte direkt am Zielsystem provoziert und quantifiziert. Um Messrauschen durch nicht-deterministische Hardware-Verhalten auszuschließen, werden auf dem Host-System tiefe C-States sowie dynamische Fre-

quenzskalierung (Intel Turbo Boost, CPU-Governor) deaktiviert. Die Messungen erfassen Durchsatz und Latenz des Opfer-Systems sowohl unter isolierter Baseline-Last als auch unter gleichzeitiger Störlast durch ein Angreifer-System. Das Delta beider Messreihen quantifiziert den Isolationsbruch.

e) *Praktische Demonstration:* Im Rahmen des Proof of Concept (PoC) werden zwei Debian-Gastsysteme — ein Angreifer und ein Opfer — über striktes CPU-Pinning (CPU Affinity) auf identische physische P-Cores des Proxmox-Hosts fixiert, um die erzwungene LLC-Konutzung sicherzustellen. Das Angreifer-System sättigt den LLC gezielt durch `stress-ng --cache 2 --cache-level 3`. Das Opfer-System erfasst parallel mittels `sysbench` und `fio` den I/O-Durchsatz (MB/s) sowie die 95. Perzentil-Latenz (ms) als primäre Interferenzmetriken. Sollten aktuelle Hardware-Mitigierungen — insbesondere Intel RDT/CAT — die Interferenz auf ein statistisch nicht signifikantes Maß reduzieren und die Hypothese damit nicht falsifizierbar machen, wird eine methodische Fallback-Strategie aktiviert: Ein systematischer Leistungsvergleich von reiner Emulation (QEMU), Para-Virtualisierung (LXC) und hardwareunterstützter Virtualisierung (KVM) unter identischer Last und strikter Host-Kontrolle. Der Fallback liefert in diesem Fall unabhängige, empirisch verwertbare Aussagen über den inhärenten Performance-Overhead der jeweiligen Virtualisierungsparadigmen.

f) *Aufbau der Arbeit:* Abschnitt II systematisiert den Stand der Forschung zu mikroarchitektonischen Interferenz-Vektoren, Virtualisierungsparadigmen und hardware-seitigen Mitigierungsstrategien. Abschnitt III beschreibt den Versuchsaufbau, die Host-Konfiguration sowie die eingesetzten Messwerkzeuge und -methoden. Abschnitt IV präsentiert und bewertet die Ergebnisse des PoC respektive des Fallback-Benchmarks. Abschnitt V diskutiert die Befunde im Kontext der Ausgangshypothese und der referierten Literatur. Abschnitt VI fasst die zentralen Erkenntnisse zusammen und benennt weiterführende Forschungsfragen.

LITERATUR

- [1] Y. Koh, R. Knauerhase, P. Brett, M. Bowman, T. F. Wenisch, and C. Pu, “An analysis of performance interference effects in virtual environments,” in *2007 IEEE International Symposium on Performance Analysis of Systems & Software*. IEEE, 2007, pp. 200–209.
- [2] Q. Ge, Y. Yarom, D. Cock, and G. Heiser, “A survey of microarchitectural timing attacks and countermeasures on contemporary hardware,” *Journal of Cryptographic Engineering*, vol. 8, no. 1, pp. 1–27, 2018.
- [3] R. Chandramouli, “Security recommendations for server-based hypervisor platforms,” National Institute of Standards and Technology, NIST Special Publication 800-125A, 2014.

ANHANG: ERSTELLUNGS-DOKUMENTATION

Dieser Anhang dokumentiert den KI-gestützten Erstellungsprozess gemäß den Anforderungen der Lehrveranstaltung. Das verwendete KI-Werkzeug ist Claude Sonnet 4.6 (Anthropic, Stand: Mai 2026).

A. Verwendeter System-Kontext

Der folgende System-Kontext wurde dem KI-Werkzeug als persistente Instruktionsgrundlage übergeben. Er legt Projektziel, Hardware-Szenario, Methodik und Fallback-Strategie fest.

A.1 Projektziel und Rahmenbedingungen

- **Format:** IEEE Conference Format (`\documentclass[conference]{IEEEtran}`)
- **Sprache:** Deutsch (wissenschaftlicher Standard)
- **Kernaufgabe:** Untersuchung der Isolation in Virtualisierungslösungen mit Fokus auf Hardware-Sicherheit

A.2 Erweiterte Aufgabenstellung (Fokus: Noisy-Neighbor)

Das Projekt erweitert die ursprüngliche Basisaufgabe (Vergleich von Virtualisierungsparadigmen) um eine spezifische Untersuchung mikroarchitektonischer Ressourcen-Interferenzen. Ziel ist die Falsifikation der Hypothese, dass historische Angriffsvektoren auf moderner Hardware (Intel Raptor Lake) durch aktuelle Isolationsmechanismen (Intel RDT/CAT) vollständig neutralisiert werden.

A.3 Hardware-Szenario und Host-Konfiguration

- **Host:** Proxmox VE auf Intel Raptor Lake (hybride P-Core/E-Core Architektur)
- **Shared Resource:** Last Level Cache (LLC), physisch geteilt von P-Cores
- **Host-Determinismus (zwingend):**
 - Deep C-States deaktivieren (Kernel-Parameter):
`intel_idle.max_cstate=1 processor.max_cstate=1 idle=poll`
 - Intel Turbo Boost deaktivieren:
`echo 1 > /sys/devices/system/cpu/intel_pstate/no_turbo`
 - CPU-Governor auf performance fixieren

A.4 Methodik und Proof of Concept (PoC)

- **Setup:** Zwei identische Debian-Instanzen (Attacker & Victim)
- **Isolations-Bruch:** CPU-Pinning beider Instanzen auf denselben physischen P-Core (CPU Affinity in Proxmox)
- **Metriken:** Durchsatz (MB/s) und 95. Perzentil-Latenz (ms) via `sysbench` und `fio` auf dem Victim-System; Attacker läuft `stress-ng --cache 2 --cache-level 3`

A.5 Fallback-Strategie

Greift, wenn keine messbaren Interferenzen auftreten:

- **Thema:** Vergleich von Emulation (QEMU), Para-Virtualisierung (LXC) und hardwareunterstützter Virtualisierung (KVM)
- **Fokus:** Leistungsfähigkeit bei Rechen- und I/O-Operationen unter strikter Host-Kontrolle

B. Verwendeter Prompt

Der folgende Prompt wurde zur Generierung der Einleitung eingesetzt. Das KI-Werkzeug erhielt diesen Prompt zusammen mit dem unter A. dokumentierten System-Kontext sowie den Quelldateien `expose_hardware_security.tex`, `literatur_recherche_draft.tex` und `references.bib`.

Rollenprofil: Wissenschaftlicher Autor im Fachbereich IT-Sicherheit und Hardware-Architektur.

Aufgabe: Erstellung eines inhaltlichen Entwurfs der Einleitung (Ausgabe als Markdown-Dokument `Einleitung.md`) für eine wissenschaftliche Hausarbeit im IEEE-Format. Die inhaltliche Struktur muss strikt und sequenziell den folgenden sechs Leitfragen folgen.

Eingabedaten (zwingend zu berücksichtigen):

- System-Kontext: `ki_agent_context_projekt.md`
- Projektskizze: `project/expose/expose_hardware_security.tex`
- Stand der Forschung: `paper/literatur_recherche_draft.tex`

- Literaturdatenbank: `paper/references.bib`

Inhaltliche Strukturierung (Absatzweise abzuarbeiten):

- 1) **Worum geht es? (Thematische Einführung):** Sachliche Einführung in Virtualisierung, logische Isolation und die physische Teilung mikroarchitektonischer Ressourcen (insbesondere Caches).
- 2) **Warum ist das Thema relevant? (Gefährdungspotenzial & Aktualität):** Einordnung der Relevanz für moderne Cloud-Infrastrukturen. Der Noisy-Neighbor-Effekt als systemische Schwachstelle (Leistungsdegradation, Timing-Angriffe, Denial-of-Service auf Hardware-Ebene).
- 3) **Was ist das Ziel dieser Hausarbeit? (Zielsetzung & Problemstellung):** Falsifikation der Hypothese, dass historische Interferenz-Vektoren auf aktuellen, hardware-mitigierten Architekturen (Intel Raptor Lake) unter produktiven Hypervisor-Bedingungen (Proxmox VE) weiterhin quantifizierbare Isolationsbrüche verursachen.
- 4) **Was ist der Lösungsansatz?:** Empirische Quantifizierung der Performance-Isolation unter kontrollierter Störlast.
- 5) **Was wird im praktischen Teil demonstriert?:** Proof of Concept: CPU-Pinning auf physische P-Cores, LLC-Sättigung via `stress-ng`, Baseline- und Degradationsmessung via `sysbench` und `fio`. Zwingende Integration der Fallback-Strategie (Leistungsvergleich QEMU/LXC/KVM), falls Intel RDT den Isolationsbruch neutralisiert.
- 6) **Wie ist die Gliederung der Arbeit?:** Rein deskriptiver Ausblick auf die nachfolgenden Kapitel (Stand der Forschung, Methodik, Evaluation, Fazit).

Stilistische und formale Vorgaben:

- Sprache: Wissenschaftliches, objektives Deutsch
- Terminologie: Korrekte Fachsprache (IT-Security / Hardware-Architektur), keine unnötigen Vereinfachungen
- Wissenschaftlicher Standard: Differenzierung zwischen Fakt (Architekturgegebenheiten) und Hypothese
- Formatierung: Ausgabe als validierbarer Markdown-Codeblock, keine Meta-Kommentare
- Referenzierung: Platzhalter im `\cite{}`-Format, korrespondierend zu den Einträgen in der `references.bib`